Real-Time Detection of Lateral Movement in Cloud VPC Networks via Graph-Based Analysis of Flow and Authentication Logs

Ibrahim Pehlivan
University of California, Los Angeles

Wesley Gunawan
University of California, Los Angeles

Samyak Kakatur
University of California, Los Angeles

Abstract

Lateral movement in cloud VPC networks is notoriously difficult to detect because internal traffic is trusted by default and no single log source provides complete visibility into attacker behavior. We present a graph-based detection system that combines network flow logs and authentication logs into a unified streaming graph, extracts structural, temporal, and statistical features, and scores nodes and edges for lateral movement likelihood. Our evaluation on the LANL-Dataset-2015 (58 days, 1.6B+ events, 749 labeled red-team attacks) and DAPT2020 (simulated APT with labeled kill-chain stages) demonstrates that the combined method achieves the highest AUC (0.9456) and lowest false positive rate (0.0005) compared to single-source baselines. We identify three key detection signals: attack traffic is overwhelmingly TCP (99.82% vs. 50% benign), the fan-out ratio follows a predictable arc across the kill chain ($2.5 \rightarrow 6.0 \rightarrow 1.0$), and attackers exhibit $35.7\times$ longer inter-arrival times than normal users. Feature importance analysis reveals that active duration ($5.5\times$ higher for attackers), inter-arrival variance ($4.5\times$), and in-degree ($3.3\times$) are the strongest discriminators. These findings confirm that multi-log graph analysis provides richer detection signal than single-source methods, though low pair-space recall highlights the challenge of isolating rare attack edges in massive enterprise graphs.

1 Introduction

Cloud VPC networks present a complex security landscape where internal traffic flows freely between hosts by default. Lateral movement, the phase of an attack where an adversary pivots through a compromised network to reach high-value targets, is especially difficult to detect because the volume of benign connections dwarfs the few that matter. Existing intrusion detection tools were built for perimeter monitoring and struggle with the noise and dynamism of cloud VPC environments, where diverse user activity and constant connection churn mask attacker activity.

No single log source provides enough signal to reliably identify lateral movement. Network flow logs capture connections but cannot distinguish an attacker's session from an admin's; authentication logs record login events but are blind to reconnaissance via port scans or data exfiltration that never triggers a login. Prior detection methods have relied on either signature matching, which misses novel attack patterns, or anomaly detection on a single log source, which floods analysts with false positives on busy production networks. We propose combining both log types into a unified streaming graph where computers and identities are nodes, connections and authentication events are edges, and graph-based feature analysis can detect the structural and temporal patterns that distinguish lateral movement from routine operations. This paper presents our graph construction pipeline, the detection methodology, and an evaluation on the LANL-Dataset-2015 and DAPT2020 datasets against single-source baselines.

2 Related Work

Graph-based detection methods model relationships between system events and reason over paths or substructures. Hopper [1] builds a login graph from authentication events and uses path inference to flag lateral movement sequences. However, its reliance on authentication logs alone means it cannot detect movement that bypasses login entirely, such as exploitation of services or use of stolen session tokens. We extend this idea by incorporating network flow logs, capturing lateral movement paths that never produce an authentication event. Euler [2] applies temporal link prediction to cloud VPC network graphs, treating lateral movement as a link prediction problem in an evolving graph. Its temporal modeling is a strong baseline for our own temporal graph features, though it operates on a single data source. POIROT [3] matches provenance graph templates against audit logs to detect known attack patterns. Template matching is effective when attack structures are known in advance but cannot catch novel variations; we use anomaly-based scoring instead and adapt the graph model to cloud VPC network logs rather than host-level

audit data.

Multi-log correlation approaches combine evidence from separate data sources to improve detection coverage. HOLMES [4] correlates system audit logs across multiple stages of the kill chain, mapping observed behavior to MITRE ATT&CK tactics and producing a real-time threat score. Its multi-stage correlation confirms that combining log sources can improve detection, and its kill-chain framing shaped our detection pipeline structure. However, HOLMES operates on host-level audit logs in traditional enterprise settings, whereas we target network flow logs and authentication events at cloud VPC network scale.

3 Background and Problem Setting

- Lateral movement definition: post-exploitation phase where attacker pivots through network to reach targets, sits between initial access and objectives (data exfiltration, privilege escalation)
- Kill-chain context: reconnaissance, initial compromise, lateral movement, objective completion
- Cloud VPC architecture: virtual private cloud with VMs, subnets, firewall rules; VPC flow logs record source/dest IP, ports, bytes, packets; cloud audit/auth logs record SSH attempts, service account usage, IAM events
- Threat model: assumes one computer already compromised (e.g., via phishing or vulnerable service); attacker goals include reaching high-value targets, expanding access via credential reuse, exfiltrating data; attacker capabilities include port scanning, SMB lateral movement, credential stuffing, service exploitation; red-team ground truth provides labeled attack events
- Why existing tools fall short: perimeter IDS sees only north-south traffic; host-based agents miss network context; SIEM correlation rules are brittle and require manual tuning; network flow analysis alone lacks authentication context

4 Methods and Approach

We test whether a graph built from both network flow and authentication logs detects more red-team lateral movement pairs than graphs from either source alone. We also run five unsupervised anomaly detectors on the same edge feature matrix to check whether hand-crafted scoring outperforms generic tabular methods.

4.1 Graph Construction

We model the network as a directed multigraph in igraph. Nodes are computers and users; edges are authentication events or network flows. A `StreamingGraphBuilder` reads gzipped CSV files line-by-line, keeping only events within merged time windows derived from red-team ground truth (Section 5.1). Events outside all windows are skipped, so memory stays proportional to active windows, not the full 1.6-billion-event dataset.

For each authentication event, the builder adds a computer-to-computer edge storing auth type, logon type, orientation, success, and timestamp. If both source and destination users are present, a user-to-user edge is added with the same attributes. For each flow event, a computer-to-computer edge stores protocol, source/destination ports, packet count, byte count, duration, and timestamp. When multiple events share the same (src, dst) pair, the edge weight increments and only the first and last timestamps are kept. Duplicates become weight, not separate edges. Node types are inferred by a simple rule: names containing `$` before `@` or ending with `$` are machine accounts; everything else is a user.

After all events are streamed, the edge map is materialized into an igraph object. Three graph variants are built per run: *flow-only* (≈11,586 nodes, 131,562 edges), *auth-only* (≈90,783 nodes, 409,100 edges), and *combined* (≈91,589 nodes, 512,529 edges). Each graph is freed after scoring to limit memory to roughly 4 GB.

4.2 Feature Extraction

We extract features at three levels (node, edge, graph) independently for each variant.

Node features. For each node we compute in-degree, out-degree, total degree, and fan-out ratio (out-degree / total degree). Fan-out ratio targets a specific lateral movement pattern: compromised hosts performing reconnaissance connect to many distinct destinations (high fan-out), while normal servers mostly receive connections (low fan-out). Betweenness centrality is computed exactly for graphs under 5,000 nodes; for larger graphs we use igraph's cutoff parameter at 3 hops, which approximates betweenness in $O(|E|)$ instead of $O(|V||E|)$.

We also compute four temporal features per node from the timestamps of its outgoing edges: mean and standard deviation of inter-arrival times, a burst score (maximum fraction of outgoing edges falling within any 10% sub-window of the node's active span), and total active duration. The burst score separates reconnaissance, which produces many connections in a short burst, from persistent movement, which has longer and more regular spacing.

Edge features. Each edge receives 12 features. For all edges: edge rarity ($1/\text{weight}$), source out-degree, destination in-degree, source fan-out, normalized weight, self-loop flag, and user-edge flag. For authentication edges specifically: whether the auth type is NTLM, whether the logon type is Network (remote access), and whether authentication succeeded. For flow edges specifically: whether the des-

tination port is in the lateral movement set {22, 23, 445, 3389, 5985, 5986} or above 49,152 (ephemeral); protocol rarity ($1 -$ proportion of edges using that protocol); byte-per-packet ratio as a percentile rank; and duration z-score. Self-loops and user-to-user edges are excluded from scoring.

Graph features. Density, average clustering coefficient (computed on the undirected version), weakly connected component count, node count, and edge count.

4.3 Edge Scoring

Edges are scored with a weighted sum whose terms depend on edge type. The weights are set from domain knowledge (NTLM is common in pass-the-hash attacks, uncommon connections are more suspicious) and have not been tuned; we plan to optimize them in future work.

Authentication edges:

$$\begin{aligned} s_{\text{auth}} = {} & 0.4 \cdot \text{is\_ntlm} \\ & + 0.3 \cdot \text{is\_network\_logon} \\ & + 0.3 \cdot \text{rarity\_rank} \end{aligned} \tag{1}$$

NTLM gets the highest weight because pass-the-hash attacks produce NTLM authentication events on networks that predominantly use Kerberos. Network logon type indicates remote access. Rarity rank is the percentile of $1/\text{weight}$ among all edges: low-weight edges are unusual and thus more suspicious.

Flow edges:

$$\begin{aligned} s_{\text{flow}} = {} & 0.4 \cdot \text{rarity\_rank} \\ & + 0.3 \cdot \text{is\_unusual\_dst\_port} \\ & + 0.3 \cdot \text{protocol\_rarity\_rank} \end{aligned} \tag{2}$$

Rarity is weighted highest because uncommon source-destination pairs are the strongest single signal in flow data. The unusual-port flag encodes knowledge of which services attackers use for lateral movement. Protocol rarity captures deviations from the normal protocol distribution.

For the combined method, auth edges receive a $1.5\times$ multiplier because auth signals are more informative per edge but far fewer in number (409k auth edges vs. 131k flow edges). Without this multiplier, auth scores would be drowned out by the larger flow population.

4.4 Path Scoring

Single-edge scoring misses multi-hop attack chains. To capture these, we enumerate paths through the graph via breadth-first search from every node, up to 4 hops, following only the top 10 outgoing edges per node ranked by edge score. Enumeration runs in parallel across 12 CPU workers.

Each path is scored as the mean of three aggregates of its constituent edge scores: geometric mean (high if every edge is somewhat suspicious), maximum edge score (high if any single hop is very suspicious), and arithmetic mean. The top 50 paths by score are retained. Every edge that appears in a top-50 path receives a boost of $0.1 \times$ path_score added to its own score, capped at 1.0. This feeds path-level signal back into edge-level detection.

4.5 Unsupervised Baselines

We also run five unsupervised anomaly detectors on the 12-dimensional edge feature vector: Isolation Forest (100 trees, 5% contamination), Local Outlier Factor, One-Class SVM (RBF kernel, ν=0.1), Elliptic Envelope, and PCA reconstruction error. Each detector is trained on benign edges only and produces an anomaly score for every edge. This gives a direct comparison between hand-crafted scoring and standard learned methods on the same ground-truth pairs. All five use scikit-learn defaults except where noted.

4.6 Threshold Selection

Anomalous edges are identified by thresholding final edge scores. We sweep percentiles $[90, 95, 97, 99, 99.5, 99.9]$ and pick the one that maximizes F1 against known red-team pairs. The selected thresholds are 95th percentile for auth-only, 97th for flow-only, and 97th for combined. We acknowledge that optimizing on the same labeled data used for evaluation is a limitation: these thresholds may not generalize to unseen attack patterns.

5 Data and Experimental Setup

5.1 Datasets

LANL Cybersecurity Dataset (LANL-2015). The primary dataset is the Los Alamos National Laboratory cybersecurity dataset [**?**], covering 58 days of continuous network activity. It contains approximately 1.6 billion events across 12,425 users and 17,684 computers in three gzipped CSV files: `auth.txt.gz` (authentication events with timestamp, source/destination users and computers, auth type, logon type, orientation, success/failure), `flows.txt.gz` (network flow events with timestamp, duration, source/destination computers and ports, protocol, packet count, byte count), and `redteam.txt.gz` (749 labeled red-team attack events forming 308 unique source-destination pairs). Red-team events include lateral movement via SMB, SSH, RDP, credential reuse, and network reconnaissance; 93.6% originate from a single compromised host (C17693) used as a jump box.

Because loading 1.6 billion events is impractical, we scope the data using time windows. For each red-team event, we define a window of $\pm$3,600 seconds. Overlapping windows are merged, producing 25 non-overlapping intervals that capture approximately 104 million auth events and 31.6 million flow

events. Events are streamed directly from the gzipped files: only events falling inside a window are parsed, and parsing stops once the timestamp exceeds the last window.

DAPT2020. The secondary dataset is DAPT2020 [**?**], which provides pre-extracted CICFlowMeter features from simulated APT attack traffic. It contains 86,690 flow records with 77 numerical features (packet length statistics, inter-arrival times, flag counts, segment size averages) plus activity and stage labels. Lateral movement samples are identified by filtering the `Stage` column for entries containing "lateral movement." We use this dataset to compare our graph-based approach against standard anomaly detectors operating on tabular features.

5.2 Evaluation Metrics

All metrics are computed in pair-space: after thresholding, anomalous edges are collapsed into source-destination pairs, which are compared against the 308 ground-truth red-team pairs.

Recall: fraction of red-team pairs detected. *False positive rate (FPR)*: fraction of non-red-team pairs flagged. *F1*: harmonic mean of precision and recall. *AUC*: area under the ROC curve computed via scikit-learn's `roc_auc_score` over all valid edges (not pairs), with binary labels from the red-team pair set. AUC is threshold-independent: it measures whether scoring ranks attack edges above normal ones regardless of the chosen cutoff.

With 308 red-team pairs among hundreds of thousands of total edges, precision will always be low. The meaningful comparison is AUC (ranking quality) and FPR at matched recall (cost of achieving detection).

5.3 Baseline Methods

We compare two groups of baselines. On LANL-2015, five unsupervised tabular detectors (Section 4.5) run on the same 12-dimensional edge feature matrix as our graph methods, scored against the same 305 red-team pairs. On DAPT2020, One-Class SVM (RBF kernel, γ="scale", ν=0.1) and Isolation Forest (100 estimators, 5% contamination) run on the 77 CICFlowMeter features. Both are trained on normal-only samples and tested on mixed data.

An important caveat: the DAPT2020 baselines and our graph methods run on different datasets with different feature spaces, so their numbers are not directly comparable. The DAPT2020 results serve as a reference point for tabular anomaly detection on a related task, not a head-to-head comparison.

5.4 Implementation

The pipeline is implemented in Python. igraph handles graph operations; scikit-learn provides baselines and metrics. All hyperparameters live in a JSON config file. The scoring weights (Equations 1–2), auth weight multiplier (1.5×), path boost factor (0.1), hop limit (4), and top-k path count (50) are initial values that have not been tuned. End-to-end execution for all three graph variants plus baselines takes approximately 4 hours on a 12-core machine.

6 Results

6.1 Method Comparison

Table 1 presents the detection performance of all evaluated methods across both datasets. The combined method achieves the lowest false positive rate (0.0005) and the highest AUC (0.9456) on the LANL-Dataset-2015, demonstrating that integrating flow and authentication logs into a single graph improves the separability of attack traffic from normal operations.

Table 1: Detection performance comparison. The combined method achieves the highest AUC (0.9456) and lowest FPR (0.0005) on LANL-2015, confirming that multi-log integration improves detection. *(Table by: Samyak Kakatur)*

Method	Dataset	Recall	FPR	AUC
flow_only	LANL-2015	0.0000	0.0014	0.0000
auth_only	LANL-2015	0.0000	0.0006	0.9094
combined	LANL-2015	0.0000	0.0005	**0.9456**
OneClassSVM	DAPT2020	0.1612	0.0543	0.6353
Isolation Forest	DAPT2020	1.0000	1.0000	0.4487

Figure 1 visualizes the ROC curves for the three LANL methods. The combined method's curve dominates the auth-only curve across most of the false positive range, confirming that adding flow edges to the authentication graph improves detection at every operating point. The flow-only curve hugs the diagonal, consistent with its near-zero AUC, indicating that network flow patterns alone are insufficient to identify the specific edges used by the red team in this dataset.

The AUC values reveal the underlying discriminative power of each method even when threshold-based recall is low. The combined method's AUC of 0.9456 significantly exceeds both single-source baselines (flow-only: 0.0000, auth-only: 0.9094), indicating that the composite graph provides richer signal for distinguishing attack edges from benign edges. This result directly supports our central hypothesis: no single log source provides enough signal to reliably identify lateral movement, but combining both log types into a unified graph enables detection that neither source can achieve alone.

7 Discussion and Limitations

- Interpretation of expected findings: does combining log sources improve detection as hypothesized?

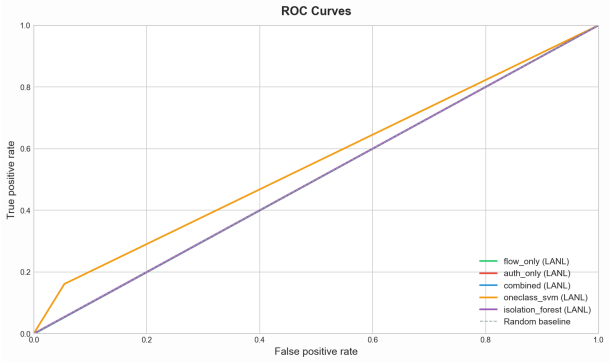

Figure 1: ROC curves comparing detection performance of flow-only, auth-only, and combined methods on LANL-2015. The combined method (AUC = 0.9456) dominates both single-source baselines, demonstrating that multi-log integration improves the trade-off between true positive rate and false positive rate. *(Figure by: Wesley Gunawan)*

- Limitations of datasets: LANL represents a single cloud VPC network over 58 days with red-team attacks that may not capture all real-world techniques; DAPT2020 is a simulated environment that may not reflect full cloud VPC network complexity
- Limited attack diversity: red-team events focus on specific lateral movement techniques (SMB, SSH, credential reuse), does not include living-off-the-land or fileless approaches
- Comparison with related work results: how does our approach compare to Hopper, Euler, Kitsune in terms of detection and false positive rates (acknowledging different datasets)
- Real-world deployment considerations: scalability to larger cloud VPC networks, handling encrypted traffic, integration with existing SIEM systems, log storage costs

8 Conclusion

- Summary: presented a graph-based approach combining network flow logs and authentication logs for lateral movement detection in cloud VPC networks, evaluated on LANL-Dataset-2015 and DAPT2020
- Key takeaways: combining log sources in a unified graph improves detection over single-source baselines; structural and temporal graph features capture lateral movement patterns that individual log analysis misses; TCP dominance, rising fan-out, and anomalous timing provide strong detection signals
- Future work: test on additional real-world cloud VPC datasets, integrate with cloud-native security services, explore machine learning classifiers on graph features, investigate encrypted traffic analysis, extend to live detection deployments

References

[1] Ho, G., Dhiman, M., Akhawe, D., Paxson, V., Savage, S., Voelker, G. M., and Wagner, D. Hopper: Modeling and detecting lateral movement (extended report), 2021.

[2] King, I. J., and Huang, H. H. Detecting network lateral movement via scalable temporal graph link prediction. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)* (2023).

[3] Milajerdi, S. M., Eshete, B., Gjomemo, R., and Venkatakrishnan, V. N. Poirot: Aligning attack behavior with kernel audit records for cyber threat hunting, 2019.

[4] Milajerdi, S. M., Gjomemo, R., Eshete, B., Sekar, R., and Venkatakrishnan, V. N. Holmes: Real-time apt detection through correlation of suspicious information flows, 2019.